

# The Thomas algorithm for tr-diagonal systems

ANDREA PROSPERETTI

## 1 Discretization of model equations

Let us consider the simple equation

$$\frac{d^2 u}{dx^2} = f(x) \quad (1.1)$$

in the interval  $0 < x < L$  subject to boundary conditions

$$u(0) = a, \quad u(L) = b. \quad (1.2)$$

We start by dividing the domain of interest in  $N + 1$  equal segments, each of length  $h$ , by introducing  $N$  equi-spaced points so that  $h = L/(N + 1)$ . Let us denote by  $x_j = jh$  the positions of these points and let us write  $u_j = u(x_j)$ ,  $f_j = f(x_j)$  for brevity. Evidently  $x_0 = 0 \cdot h = 0$  and  $x_{N+1} = (N + 1)h = L$ . At the generic point  $x_j$ , with  $j \neq 0$ ,  $j \neq N + 1$ , we approximate the second derivative in (1.1) by writing

$$\frac{u_{j+1} - 2u_j + u_{j-1}}{h^2} = f_j, \quad (1.3)$$

from which

$$u_{j-1} - 2u_j + u_{j+1} = h^2 f_j. \quad (1.4)$$

For  $j = 1$  we take  $u_{j-1} = a$  to the right-hand side since it is known:

$$-2u_1 + u_2 = h^2 f_1 - a, \quad (1.5)$$

and, similarly, for  $j = N$  we take  $u_{j+1} = b$  to the right-hand side

$$u_{N-1} - 2u_N = h^2 f_N - b. \quad (1.6)$$

In this way we have generated the linear algebraic system

$$-2u_1 + u_2 = h^2 f_1 - a \quad (1.7a)$$

$$u_1 - 2u_2 + u_3 = h^2 f_2 \quad (1.7b)$$

$$u_2 - 2u_3 + u_4 = h^2 f_3 \quad (1.7c)$$

...

$$u_{N-2} - 2u_{N-1} + u_N = h^2 f_3 \quad (1.7d)$$

$$u_{N-1} - 2u_N = h^2 f_N - b. \quad (1.7e)$$

The problem of solving the Poisson equation has then been reduced to the problem of solving this linear system. This is a simple example of one of the many ways in which linear systems become associated with the numerical solution of differential equations.

For the the one-dimensional Helmholtz equation

$$\frac{d^2 u}{dx^2} - ku = f(x) \quad (1.8)$$

in place of (1.3) we would have

$$\frac{u_{j-1} - 2u_j + u_{j+1}}{h^2} - ku_j = f_j, \quad (1.9)$$

Upon rearranging this becomes

$$u_{j-1} - (2 + kh^2)u_j + u_{j+1} = h^2 f_j, \quad (1.10)$$

which differs only slightly from (1.4). Boundary conditions at both end-points are also needed and they are handled in the same way.

## 2 Numerical solution of tri-diagonal linear systems

We consider a linear algebraic system of the form

$$a_1 u_1 + c_1 u_2 = f_1 \quad (2.1a)$$

$$b_2 u_1 + a_2 u_2 + c_2 u_3 = f_2 \quad (2.1b)$$

...

$$b_{N-1} u_{N-2} + a_{N-1} u_{N-1} + c_{N-1} u_N = f_{N-1} \quad (2.1c)$$

$$b_N u_{N-1} + a_N u_N = f_N. \quad (2.1d)$$

We re-write this system in matrix form as

$$\begin{pmatrix} a_1 & c_1 & 0 & 0 & 0 & \dots & 0 & 0 \\ b_2 & a_2 & c_2 & 0 & 0 & \dots & 0 & 0 \\ 0 & b_3 & a_3 & c_3 & 0 & \dots & 0 & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 0 & 0 & 0 & b_{N-1} & a_{N-1} & c_{N-1} \\ 0 & 0 & 0 & 0 & 0 & 0 & b_N & a_N \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \\ \dots \\ \dots \\ \dots \\ u_{N-1} \\ u_N \end{pmatrix} = \begin{pmatrix} f_1 \\ f_2 \\ f_3 \\ \dots \\ \dots \\ \dots \\ f_{N-1} \\ f_N \end{pmatrix} \quad (2.2)$$

It will be noticed that the matrix has non-zero elements only on the main diagonal and on the two adjacent subdiagonals. This is a very special form which greatly facilitates the solution as we will see. Matrices of this special type are called *tri-diagonal matrices*.

To explain the method let us consider a  $3 \times 3$  system:

$$\begin{pmatrix} a_1 & c_1 & 0 \\ b_2 & a_2 & c_2 \\ 0 & b_3 & a_3 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} = \begin{pmatrix} f_1 \\ f_2 \\ f_3 \end{pmatrix} \quad (2.3)$$

i.e.

$$a_1 u_1 + c_1 u_2 = f_1 \quad (2.4)$$

$$b_1 u_1 + a_2 u_2 + c_2 u_3 = f_2 \quad (2.5)$$

$$b_3 u_2 + a_3 u_3 = f_3 \quad (2.6)$$

To eliminate  $u_1$  between the first two equations we form the combination  $(2.5) - (b_2/a_1) \times (2.4)$  to find

$$(b_1 u_1 + a_2 u_2 + c_2 u_3) - \frac{b_2}{a_1} (a_1 u_1 + c_1 u_2) = f_2 - \frac{b_2}{a_1} f_1 \quad (2.7)$$

which reduces to

$$\left( a_2 - \frac{b_2 c_1}{a_1} \right) u_2 + c_2 u_3 = f_2 - \frac{b_2}{a_1} f_1 \quad (2.8)$$

Define

$$\alpha_2 = a_2 - \frac{b_2 c_1}{a_1}, \quad g_2 = f_2 - \frac{b_2}{a_1} f_1 \quad (2.9)$$

Then we are left with the two equations

$$\alpha_2 u_2 + c_2 u_3 = g_2 \quad (2.10)$$

$$b_3 u_2 + a_3 u_3 = f_3 \quad (2.11)$$

We operate as before forming  $(2.11) - (b_3/\alpha_2) \times (2.10)$  to find

$$(b_3u_2 + a_3u_3) - \frac{b_3}{\alpha_2}(\alpha_2u_2 + c_2u_3) = f_3 - \frac{b_3}{\alpha_2}g_2 \quad (2.12)$$

from which

$$\left(a_3 - \frac{b_3}{\alpha_2}c_2\right)u_3 = f_3 - \frac{b_3}{\alpha_2}g_2 \quad (2.13)$$

which, with

$$\alpha_3 = a_3 - \frac{b_3}{\alpha_2}c_2, \quad g_3 = f_3 - \frac{b_3}{\alpha_2}g_2 \quad (2.14)$$

we can write as

$$\alpha_3u_3 = g_3 \quad (2.15)$$

which is easily solved to give  $g_3/\alpha_3$ . At this point we go back to (2.10) to find

$$u_2 = \frac{g_2 - c_2u_3}{\alpha_2} \quad (2.16)$$

and to (2.4) to find

$$a_1u_1 + c_1u_2 = f_1 \quad (2.17)$$

from which we can calculate  $u_1$ .

In summary, if we set

$$\alpha_1 = a_1, \quad g_1 = f_1, \quad (2.18)$$

we see that

$$\alpha_2 = a_2 - \frac{b_2c_1}{\alpha_1}, \quad g_2 = f_2 - \frac{b_2}{\alpha_1}g_1, \quad (2.19)$$

$$\alpha_3 = a_3 - \frac{b_3}{\alpha_2}c_2, \quad g_3 = f_3 - \frac{b_3}{\alpha_2}g_2, \quad (2.20)$$

after which we find the solution as

$$u_3 = \frac{g_3}{\alpha_3}, \quad (2.21)$$

$$u_2 = \frac{g_2 - c_2u_3}{\alpha_2} \quad (2.22)$$

$$u_1 = \frac{g_1 - c_1u_2}{\alpha_1} \quad (2.23)$$

There is a clear pattern here. For the general case of an  $N \times N$  system it goes like this:

$$\alpha_1 = a_1, \quad g_1 = f_1, \quad (2.24)$$

$$\alpha_j = a_j - \frac{b_jc_{j-1}}{\alpha_{j-1}}, \quad g_j = f_j - \frac{b_j}{\alpha_{j-1}}g_{j-1} \quad \text{for } j = 2, 3, \dots, N \quad (2.25)$$

after which

$$u_N = \frac{g_N}{\alpha_N}, \quad (2.26)$$

and

$$u_j = \frac{g_j - c_ju_{j+1}}{\alpha_j} \quad \text{for } j = N-1, N-2, \dots, 1. \quad (2.27)$$

This is the so-called *Thomas* or *tri-diagonal algorithm*. A pseudo-code for this algorithm looks as follows:

```

 $\alpha_1 = a_1$ 
 $g_1 = f_1$ 

for  $j = 2, N$ 
 $\alpha_j = a_j - (b_j/\alpha_{j-1})c_{j-1}$ 
 $g_j = f_j - (b_j/\alpha_{j-1})g_{j-1}$ 
end

 $u_N = g_N/\alpha_N$ 

for  $k = 1, N-1$ 
 $u_{N-k} = (g_{N-k} - c_{N-k}u_{N-k+1})/\alpha_{N-k}$ 
end

```

It may be noted that, since  $a_j$  and  $f_j$  are never re-used after they are used to calculate  $\alpha_j$  and  $g_j$ , they can actually be over-written. Thus, in place of  $\alpha_j = a_j - (b_j/\alpha_{j-1})c_{j-1}$  and  $g_j = f_j - (b_j/\alpha_{j-1})g_{j-1}$  we can simply save memory by writing  $a_j = a_j - (b_j/\alpha_{j-1})c_{j-1}$  and  $f_j = f_j - (b_j/\alpha_{j-1})g_{j-1}$ . (Note that the equal sign here is used to assign a value as in most programming languages. It does not mean equality. So for example, the meaning of  $a_j = a_j - (b_j/\alpha_{j-1})c_{j-1}$  is that the result of the calculation  $a_j - (b_j/\alpha_{j-1})c_{j-1}$  is stored in the same memory location that contained  $a_j$ .)

A careful consideration of the method just described identifies some possible problems that can arise in its application:

1. Since each step of both the direct and inverse procedures requires division by  $\alpha_j$ , to apply the method it is necessary that none of the  $\alpha_j$ 's vanish. There are theorems that give sufficient conditions for this not to happen. These theorems point to a very desirable feature of the matrix called *diagonal dominance*. The matrix of the system (2.2) is diagonally dominant if

$$|a_j| \geq |b_j| + |c_j|, \quad j = 1, 2, \dots, N, \quad (2.28)$$

with at least one of the inequalities holding in the strict sense (i.e., with at least one the “ $\geq$ ” replaced by a “ $>$ ”). For the Poisson equation, i.e., (1.9) with  $\lambda = 0$ ,  $a_j = -2$ ,  $b_j = 1$ ,  $c_j = 1$  and this condition holds with the equal sign for  $2 \leq j \leq N-1$ . For  $j = 1$   $b_1 = 0$ , and for  $j = N$  and  $c_N = 0$ , so that for these values of  $j$  the inequality holds in the strict sense. The situation is more delicate for the Helmholtz equation (i.e.,  $\lambda \neq 0$ ), where a more careful analysis may be necessary.

2. The backward recurrence relation (2.27) can cause problems because of round-off errors. For example, suppose that  $u_N$  as determined by (2.26) is affected by an error  $\epsilon_N$ , so that the quantity that is actually calculated is  $\tilde{u}_N = u_N + \epsilon_N$  in place of the “true value”  $u_N$ . Then (2.27) gives

$$\tilde{u}_{N-1} = u_{N-1} + \epsilon_{N-1} = \frac{g_{N-1} - c_{N-1}\tilde{u}_N}{\alpha_{N-1}} = \frac{g_{N-1} - c_{N-1}u_N}{\alpha_{N-1}} - \frac{c_{N-1}\epsilon_N}{\alpha_{N-1}}, \quad (2.29)$$

which shows that the error induced in the determination of  $u_{N-1}$  is given by

$$\epsilon_{N-1} = -\frac{c_{N-1}}{\alpha_{N-1}}\epsilon_N. \quad (2.30)$$

At the next step we find an error

$$\epsilon_{N-2} = -\frac{c_{N-2}}{\alpha_{N-2}}\epsilon_{N-1} = \left(-\frac{c_{N-2}}{\alpha_{N-2}}\right) \left(-\frac{c_{N-1}}{\alpha_{N-1}}\right) \epsilon_N, \quad (2.31)$$

and so forth. It is evident that, if  $c_j/\alpha_j > 1$ , the error is amplified at each step and, with a large system of equations, can grow large enough to destroy the calculation. A sufficient condition to avoid this problem is diagonal dominance, and also that none of the  $b_j$ 's and  $c_j$ 's vanish.

3. The previous conditions are only sufficient. Suppose for example that  $b_K = 0$  for a certain index  $K$ . In this case the unknown  $u_{K-1}$  drops out of the  $K$ -th equation and, since this is the only link between the  $K$ -th equation and the preceding ones, the system reduces to two smaller systems, one with unknowns  $u_1, u_2, \dots, u_{K-1}$  and one with unknowns  $u_K, u_{K+1}, \dots, u_N$ , which can be solved separately one from the other.

### 3 Neumann boundary conditions

In the case of boundary conditions of the Neumann type, it is the derivative of  $u$  at the boundary points that is prescribed. For example, for  $x = 0$ , the mathematical statement of the problem prescribes that

$$\left. \frac{du}{dx} \right|_{x=0} = v, \quad (3.1)$$

with  $v$  given, but  $u_0 = u(x = 0)$  itself unknown. A simple idea to achieve this goal is to approximate (3.1) by a forward formula:

$$\frac{u_1 - u_0}{h} \simeq v + O(h) \quad (3.2)$$

from which

$$-u_0 + u_1 = hv. \quad (3.3)$$

We can consider this as the first equation of the system and continue in the normal way.

The disadvantage of this procedure is that, while the formula for the second derivative that we use in (1.3) and the analogous Helmholtz equation is highly accurate, that used for (3.1) is of lower accuracy. Many problems are critically sensitive to boundary conditions, and this procedure runs the risk of injecting a significant error in the entire calculation unless a very fine grid is used. A better procedure might be to use, in place of (3.2), the more accurate three-point forward formula

$$\frac{4u_2 - 3u_1 - u_0}{2h} \simeq v + O(h^2) \quad (3.4)$$

with which the first equation of the system would be

$$-u_0 - 3u_1 + 4u_2 = 2hv. \quad (3.5)$$

But, if we do this, we have destroyed the tri-diagonal structure of the system as the first equation now has 3 unknowns rather than 2. A smarter way is to introduce what is called a *ghost node* at  $x = x_{-1} = -h$ . Now, in place of (3.2) we have the higher-accuracy formula

$$\frac{u_1 - u_{-1}}{2h} \simeq v + O(h^2) \quad (3.6)$$

from which  $u_{-1} = u_1 - 2hv$ . If we now write (1.10) for  $j = 0$  we have

$$u_{-1} - (2 - \lambda h^2)u_0 + u_1 = h^2 f_0 \quad (3.7)$$

and, upon expressing  $u_{-1}$  using (3.7),

$$-(2 - \lambda h^2)u_0 + 2u_1 = h^2 f_0 + 2hv, \quad (3.8)$$

which has the desired form.